Claude Code 总体全览

前言：在前面，我们学习了机器学习与深度学习，Transformer架构、RAG、Agent、多Agent联合编排、大模型微调技术、Harness等内容。今天我们就来看一下，上述内容在当今时代的最完美艺术品中：Claude Code 是怎么将上述的技术协调统一的进行组合使用的。

一、Claude Code 是什么

1.1 定义

Claude Code 是 Anthropic 官方出品的命令行 AI 编程助手。你在终端输入一句话，它就能帮你读代码、写代码、搜文件、跑命令、修 Bug、提交 PR。
从源码来看，Claude Code 是**完整的 Agent 系统**，核心能力：

1. 自主规划：自动拆分步骤、决定执行顺序
2. 工具调用：读写文件、执行命令、搜索代码、访问网页
3. 上下文管理：自动压缩长对话，避免遗忘
4. 权限控制：危险操作需用户确认，保障代码安全
5. 多 Agent 协作：支持子 Agent 并行工作
6. 记忆系统：通过 CLAUDE.md + memdir 存储规则、偏好与长期记忆

1.2 Harness 到底是什么

核心观点：**Agent = Model + Harness**

- 模型：超级聪明的大脑
- Harness：组装各类能力的框架，相当于 Agent 的身体（包含工具、记忆、权限、交互等组件）
- 核心逻辑：光有大脑无法执行任务，需搭配眼睛（读文件）、手（写文件）、保镖（权限）、记忆本（CLAUDE.md）才能完成实际工作

二、源码分层

打开 Claude Code 的 src/ 目录，按七大主题拆解（个人理解），实际每个模块含大量辅助文件。这个大家需要自己去结合源码与ai查看

三、七大模块详解

3.1 核心引擎：Agent 的 心脏

对应文件：query.ts、QueryEngine.ts
核心设计：

1. **while (true) 自动循环**：模型返回后，若需调用工具则将结果回传继续循环，无新 tool_use 时退出
2. **AsyncGenerator 异步生成器**：流式输出中间状态，实时展示执行进度
3. 解决问题：单轮 LLM 调用无法完成多步骤复杂任务

3.2 工具系统：Agent 的 手和脚

对应目录：tools/
核心特点：

1. 工具是 Agent 与真实世界交互的唯一方式，统一接口规范
2. 工具列表动态变化：随 Feature Flag、运行环境、MCP 启用状态调整
3. 标准化工具模块：输入校验、权限声明、并发安全标记、执行逻辑
4. 核心工具：BashTool、FileReadTool、FileEditTool、AgentTool、WebSearchTool 等

3.3 提示词系统：Agent 的 性格和规则

对应文件：constants/prompts.ts
核心设计：

1. 动态组装提示词：getSystemPrompt () 函数根据工具、模型、环境参数生成
2. 分段设计：支持 Prompt Cache 节省成本、模块化维护
3. 按需加载：未配置的功能（如 MCP）对应提示段自动剔除

3.4 上下文管理：Agent 的 大脑容量管理

对应目录：services/compact/
核心能力：

1. 多层组合压缩策略，解决上下文窗口超限问题
2. 自动压缩触发：对话 token 超 167000 时启动
3. 实现效果：支持 无限对话，用户无感知

3.5 权限与安全：Agent 的 保险丝

对应目录：utils/permissions/、hooks/toolPermission/
核心价值：企业级 Agent 与玩具 Agent 的核心区别

1. 多权限模式：default、acceptEdits、bypassPermissions、dontAsk、plan
2. 多层权限裁决：用户规则 → Hook 脚本 → 权限模式 → AI 分类器 → 交互确认
3. 架构级安全：每次工具调用必过权限检查，无法绕过

3.6 记忆系统：Agent 的 长期记忆

对应文件：utils/claudemd.ts、context.ts、memdir/memdir.ts
双记忆体系：

1. **指令文件（CLAUDE.md）**：用户手动维护，四层优先级规则注入
2. **自动记忆（memdir）**：系统自动提取，分记忆规则与记忆内容两条注入链路
3. 额外能力：自动读取 Git 状态作为系统上下文，无需用户手动告知分支信息

3.7 交互界面：Agent 的 脸面

对应文件：main.tsx、screens/、components/
核心实现：

1. 基于 React + Ink 库，将 React 组件渲染到终端
2. 入口文件 main.tsx：解析 CLI 参数、初始化服务、加载配置、启动交互循环

四、一次完整交互的全链路

将七大模块串联，完整呈现用户指令从输入到输出的端到端执行流程。

五、从源码中提炼的设计原则

1. 循环而非链式：模型自主决定停止时机，非预设步数
2. 工具即能力边界：模型必须通过工具间接操作文件 / 网络
3. 上下文是第一公民：重点处理上下文预算、压缩、恢复与注入
4. 提示词是可编程的：动态拼装，适配不同用户与场景
5. 安全是架构级的：权限控制内嵌循环，不可跳过

六、对比：Claude Code vs 简单 Agent

维度	简单的 Agent 实现	Claude Code
循环	单次调用或固定步数	无限循环，模型自主停止
工具	硬编码函数	标准化工具 + 条件注册 + MCP 扩展
上下文	超长截断 / 报错	多层压缩，无限对话
提示词	写死字符串	分段组装，支持缓存
权限	简陋 / 无	多层权限 + Hook+AI 分类
多 Agent	不支持	子 Agent、团队协作
记忆	无	四层 CLAUDE.md + 自动记忆
界面	基础打印	React 组件，丰富交互

小结

核心结论：**Claude Code = while (true) 循环 + 标准化工具 + 多层上下文 + 多层权限 + 分层记忆**

1. 模型只是引擎，Harness 框架是价值核心
2. 企业级 Agent 复杂度不在模型调用，而在 Harness 框架设计
3. 七大模块各司其职，共同支撑端到端自动化任务执行